J.N.T.U.H UNIVERSITY COLLEGE OF ENGINEERING HYDERABAD

(Autonomous)

KUKATPALLY, HYDERABAD - 500 085

Certificate

Certified that this is the bonafide record of the practical work done during the academic year _____ by

Name _____

Roll Number _____ Class _____

In the Laboratory of _____

of the Department of _____

Signature of the Staff Member ***Signature of the Head of the Department***

Date of Examination_____

Signature of the Examiner/s

Internal Examiner ***External Examiner*** q

J.N.T.U.H UNIVERSITY COLLEGE OF ENGINEERING HYDERABAD

(Autonomous)

KUKATPALLY, HYDERABAD - 500 085

Name _____ ***Roll Number*** _____

Class _____ ***Year*** _____

Laboratory_____

List of Experiments

S.No.	Name of the Experiment	Date of Experiment	Page Number	Marks	Remarks
1	Write a python program to compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation				
2	Implementation of Find-S Algorithm				
3	Implementation of Candidate Elimination Algorithm				
4	Implementation of Back Propagation Algorithm				
5	Implementation of K-Means Clustering Algorithm				
6	Implementation of KNN Algorithm				
7	Implementation of Decision tree using sklearn				

1.Write a python program to compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variation and Standard Deviation

```
import numpy as np
from statistics import mean, median, mode

nominal = [1,1,1,2,3,4,2,3,2,2,2,2]
print("Nominal Data")
print("Data:", nominal)
print("Mode =", mode(nominal))
print()

ordinal = [1,2,2,3,4,1,3,3,2]
print("Ordinal Data")
print("Data:", ordinal)
print("Median =", median(ordinal))
print("Mode =", mode(ordinal))
q1 = np.percentile(ordinal, 25)
q3 = np.percentile(ordinal, 75)
print("Q1 =", q1, "Q3 =", q3)
print()

discrete = [2,4,4,5,7,9,10]
print("Discrete Data")
print("Data:", discrete)
print("Mean =", mean(discrete))
print("Median =", median(discrete))
print("Mode =", mode(discrete))
print("Variance =", np.var(discrete))
print("Standard Deviation =", np.std(discrete))
q1 = np.percentile(discrete, 25)
q2 = np.percentile(discrete, 50)
```

```
q3 = np.percentile(discrete, 75)
print("Q1 =", q1, "Q2 =", q2, "Q3 =", q3)
print()

continuous = [150.5,155.2,160.8,162.1,165.9,170.3,172.6]
print("Continuous Data")
print("Data:", continuous)
print("Mean =", mean(continuous))
print("Median =", median(continuous))
print("Mode =", mode(continuous))
print("Variance =", np.var(continuous))
print("Standard Deviation =", np.std(continuous))
q1 = np.percentile(continuous, 25)
q2 = np.percentile(continuous, 50)
q3 = np.percentile(continuous, 75)
print("Q1 =", q1, "Q2 =", q2, "Q3 =", q3)
print()

interval = [10,12,15,18,20,22,25]
print("Interval Data")
print("Data:", interval)
print("Mean =", mean(interval))
print("Median =", median(interval))
print("Mode =", mode(interval))
print("Variance =", np.var(interval))
print("Standard Deviation =", np.std(interval))
q1 = np.percentile(interval, 25)
q2 = np.percentile(interval, 50)
q3 = np.percentile(interval, 75)
print("Q1 =", q1, "Q2 =", q2, "Q3 =", q3)
print()
```

```
ratio = [1,2,3,4,5,6,7,8,9,10]
print("Ratio Data")
print("Data:", ratio)
print("Mean =", mean(ratio))
print("Median =", median(ratio))
print("Mode =", mode(ratio))
print("Variance =", np.var(ratio))
print("Standard Deviation =", np.std(ratio))
q1 = np.percentile(ratio, 25)
q2 = np.percentile(ratio, 50)
q3 = np.percentile(ratio, 75)
print("Q1 =", q1, "Q2 =", q2, "Q3 =", q3)
```

OUTPUT:

```
[Running] python -u "c:\Users\Sai Naga Shreechitra\OneDrive\Desktop\AI LAB\kmc.py
Nominal Data
Data: [1, 1, 1, 2, 3, 4, 2, 3, 2, 2, 2, 2]
Mode = 2

Ordinal Data
Data: [1, 2, 2, 3, 4, 1, 3, 3, 2]
Median = 2
Mode = 2
Q1 = 2.0 Q3 = 3.0

Discrete Data
Data: [2, 4, 4, 5, 7, 9, 10]
Mean = 5.857142857142857
Median = 5
Mode = 4
Variance = 7.2653061224489806
Standard Deviation = 2.695423180587601
Q1 = 4.0 Q2 = 5.0 Q3 = 8.0

Continuous Data
Data: [150.5, 155.2, 160.8, 162.1, 165.9, 170.3, 172.6]
Mean = 162.4857142857143
Median = 162.1
Mode = 150.5
Variance = 53.53551020408166
Standard Deviation = 7.316796444078628
Q1 = 158.0 Q2 = 162.1 Q3 = 168.10000000000002

Interval Data
Data: [10, 12, 15, 18, 20, 22, 25]
Mean = 17.428571428571427
Median = 18
Mode = 10
Variance = 25.102040816326532
Standard Deviation = 5.010193690500052
Q1 = 13.5 Q2 = 18.0 Q3 = 21.0

Ratio Data
Data: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Mean = 5.5
Median = 5.5
Mode = 1
Variance = 8.25
Standard Deviation = 2.8722813232690143
Q1 = 3.25 Q2 = 5.5 Q3 = 7.75
```

2. Implementation of Find-S Algorithm

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
def find_s(dataset):
    for instance in dataset:
        if instance[-1] == 'Yes':
            hypothesis = instance[:-1]
            break
    for instance in dataset:
        if instance[-1] == 'Yes':
            for i in range(len(hypothesis)):
                if hypothesis[i] != instance[i]:
                    hypothesis[i] = '?'

    return hypothesis

final_hypothesis = find_s(data)
print("Final Hypothesis:", final_hypothesis)
```

OUTPUT

```
Final Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

3.Implementation of Candidate Elimination Algorithm

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
S = ['0'] * 6
G = [['?' for _ in range(6)]]
for instance in data:
    x, label = instance[:-1], instance[-1]
    if label == 'Yes':
        for i in range(6):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?’
        G = [
            g for g in G
            if all(g[i] == '?' or g[i] == x[i] for i in range(6))
        ]
    else:
        # Specialize general hypotheses
        G += [
            [g[i] if g[i] == x[i] else '?' for i in range(6)]
            for g in G
        ]
print("Specific hypothesis:", S)
print("General hypotheses:", G)
```

OUTPUT

```
Specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
General hypotheses: [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]
```

4. Implementation of Back Propagation Algorithm

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x):
    return x * (1 - x)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])
np.random.seed(0)
w1 = np.random.rand(2,2)
b1 = np.zeros((1,2))
w2 = np.random.rand(2,1)
b2 = np.zeros((1,1))
lr = 0.1
for _ in range(10000):
    h = sigmoid(np.dot(X, w1) + b1)
    o = sigmoid(np.dot(h, w2) + b2)
    error = y - o
    d2 = error * sigmoid_deriv(o)
    d1 = d2.dot(w2.T) * sigmoid_deriv(h)
    w2 += h.T.dot(d2) * lr
    b2 += np.sum(d2, axis=0) * lr
    w1 += X.T.dot(d1) * lr
    b1 += np.sum(d1, axis=0) * lr
print("Final Output:\n", o)
```

OUTPUT

5. Implementation of K-Means Clustering Algorithm.

```
from sklearn.cluster import KMeans
import numpy as np
X = np.array([
   [1, 2],
   [1, 4],
   [1, 0],
   [10, 2],
   [10, 4],
   [10, 0]
])
kmeans = KMeans(n_clusters=2)
kmeans.fit(X)
labels = kmeans.labels_
centers = kmeans.cluster_centers_
print("Labels:", labels)
print("Centers:", centers)
```

OUTPUT

6. Implementation of KNN Algorithm

```
data = [[2],[4],[6],[8]]
labels = ["A","A","B","B"]
test = [5]
k = 3
dist = []
for i in range(len(data)):
    d = abs(data[i][0] - test[0])
    dist.append([d, labels[i]])
for i in range(len(dist)):
    for j in range(i+1, len(dist)):
        if dist[i][0] > dist[j][0]:
            dist[i], dist[j] = dist[j], dist[i]
count = {}
for i in range(k):
    label = dist[i][1]
    if label in count:
        count[label] += 1
    else:
        count[label] = 1
res = ""
maxv = 0
for key in count:
    if count[key] > maxv:
        maxv = count[key]
        res = key
print("Prediction:", res)
```

OUTPUT:

Prediction : A

7. Implemenation of Decision Tree using sklearn and its parameter tuning

```
import math
data = [
    ["Sunny","Hot","High","Weak","No"],
    ["Sunny","Hot","High","Strong","No"],
    ["Overcast","Hot","High","Weak","Yes"],
    ["Rain","Mild","High","Weak","Yes"]
]
attributes = ["Outlook","Temperature","Humidity","Wind"]
def entropy(dataset):
    total = len(dataset)
    yes = sum(1 for row in dataset if row[-1] == "Yes")
    no = total - yes
    def calc(p):
        return 0 if p == 0 else -p * math.log2(p)
    return calc(yes/total) + calc(no/total)
def info_gain(dataset, attr_index):
    total_entropy = entropy(dataset)
    total = len(dataset)
    values = set(row[attr_index] for row in dataset)
    weighted_entropy = 0
    for v in values:
        subset = [row for row in dataset if row[attr_index] == v]
        weighted_entropy += (len(subset)/total) * entropy(subset)
    return total_entropy - weighted_entropy
print("Total Entropy:", entropy(data))
gains = []
for i in range(len(attributes)):
    gain = info_gain(data, i)
    gains.append(gain)
    print(f"Info Gain ({attributes[i]}):", gain)
best_index = gains.index(max(gains))
```

```
print("\nBest Attribute (Root Node):", attributes[best_index])
```

OUTPUT

```
Total Entropy: 1.0
Info Gain (Outlook): 1.0
Info Gain (Temperature): 0.31127812445913283
Info Gain (Humidity): 0.0
Info Gain (Wind): 0.31127812445913283

Best Attribute (Root Node): Outlook

[Done] exited with code=0 in 0.094 seconds
```